DATA ANALYTICS

DATA MANAGEMENT, DATA VIZ & TEXT MINING

Diplôme Sorbonne Data Analytics

DATA MANAGEMENT AVEC PANDAS

Introduction

De quoi parle-t-on ?

Il s’agit des techniques utilisées pour manipuler et analyser la donnée.

C’est une étape préalable à tous projets de ML réussi !

Les grandes étapes du data management

- La collecte
- La compréhension
- Le nettoyage
- La création de valeur

Présentation de Pandas

Pandas est le module **incontournable** pour la manipulation de données !

Créé en **2008** et passé en open source en **2009**, Pandas a l'ambition d'être la référence en matière de data management

"it has the broader goal of becoming the most powerful and flexible open source data analysis / manipulation tool available in any language"

Un module pas si simple

- Plusieurs manières d'atteindre un même but
- De nombreuses méthodes et attributs …

Structures de données dans Pandas

Pandas a deux structures de données principales :

- DataFrame, qui est bidimensionnel
- Série, qui est unidimensionnelle

DataFrame

	name	calories	protein	vitamins	rating
0	100% Bran	70	4	25	68.402973
1	100% Natural Bran	120	3	0	33.983679
2	All-Bran	70	4	25	59.425505
3	All-Bran with Extra Fiber	50	4	25	93.704912
4	Almond Delight	110	2	25	34.384843
5	Apple Cinnamon Cheerios	110	2	25	29.509541
6	Apple Jacks	110	2	25	33.174094
7	Basic 4	130	3	25	37.038562
8	Bran Chex	90	2	25	49.120253
9	Bran Flakes	90	3	25	53.313813

```
0     70
1    120
2     70
3     50
4    110
5    110
6    110
7    130
8     90
9     90
Name: calories, dtype: int64
```

Series

Structures de données dans Pandas

Qu’est ce qu’un DataFrame dans Pandas ?

Pandas fournit une structure de données bidimensionnelle appelée DataFrame.
• Une ligne est représentée par des étiquettes de ligne, également appelées index, qui peuvent être numériques ou chaînes de caractères.
• Une colonne est représentée par des étiquettes de colonne qui peuvent être numériques ou des chaînes.

Exemple:

index

	name	calories	protein	vitamins	rating
0	100% Bran	70	4	25	68.402973
1	100% Natural Bran	120	3	0	33.983679
2	All-Bran	70	4	25	59.425505
3	All-Bran with Extra Fiber	50	4	25	93.704912
4	Almond Delight	110	2	25	34.384843
5	Apple Cinnamon Cheerios	110	2	25	29.509541
6	Apple Jacks	110	2	25	33.174094
7	Basic 4	130	3	25	37.038562
8	Bran Chex	90	2	25	49.120253
9	Bran Flakes	90	3	25	53.313813

column labels

Structures de données dans Pandas

Un **DataFrame** Pandas est juste une **collection** d'une ou de plusieurs **series.**

Exemple :

DataFrame

	name	calories	protein	vitamins	rating
0	100% Bran	70	4	25	68.402973
1	100% Natural Bran	120	3	0	33.983679
2	All-Bran	70	4	25	59.425505
3	All-Bran with Extra Fiber	50	4	25	93.704912
4	Almond Delight	110	2	25	34.384843
5	Apple Cinnamon Cheerios	110	2	25	29.509541
6	Apple Jacks	110	2	25	33.174094
7	Basic 4	130	3	25	37.038562
8	Bran Chex	90	2	25	49.120253
9	Bran Flakes	90	3	25	53.313813

```
0     70
1    120
2     70
3     50
4    110
5    110
6    110
7    130
8     90
9     90
Name: calories, dtype: int64
```

Series

Identical

Notebook 1 : Découvrir les Séries

Création de DataFrame

1. Utilisation des lists :

On passe la liste en argument à la fonction pandas.DataFrame() qui nous renvoie un DataFrame de données.

La fonction peut prendre en paramètre les valeurs ‘index’ et ‘columns’ :

- index : Attribuer un index spécifique pour chaque ligne
- columns : Attribuer un nom à chaque colonne

Attention :

• Si non précisé, Pandas attribue automatiquement des étiquettes de ligne (index) numériques à chaque ligne du DataFrame.

• Également, si non précisé Pandas attribue des valeurs numériques comme étiquettes de colonne (columns) à chaque colonne.

Création de DataFrame

Exemple :

```
In [2]: myList = [['Apple', 'Red'],
                  ['Banana', 'Yellow'],
                  ['Orange', 'Orange']]

In [3]: myDataFrame = pd.DataFrame(myList)
        myDataFrame

Out[3]:
```

	0	1
0	Apple	Red
1	Banana	Yellow
2	Orange	Orange

Sans utilisation des paramètres

```
In [2]: myList = [['Apple', 'Red'],
                  ['Banana', 'Yellow'],
                  ['Orange', 'Orange']]

In [4]: myDataFrame = pd.DataFrame(myList, columns=['Fruit', 'Color'])
        myDataFrame

Out[4]:
```

	Fruit	Color
0	Apple	Red
1	Banana	Yellow
2	Orange	Orange

Avec utilisation du paramètre “columns”

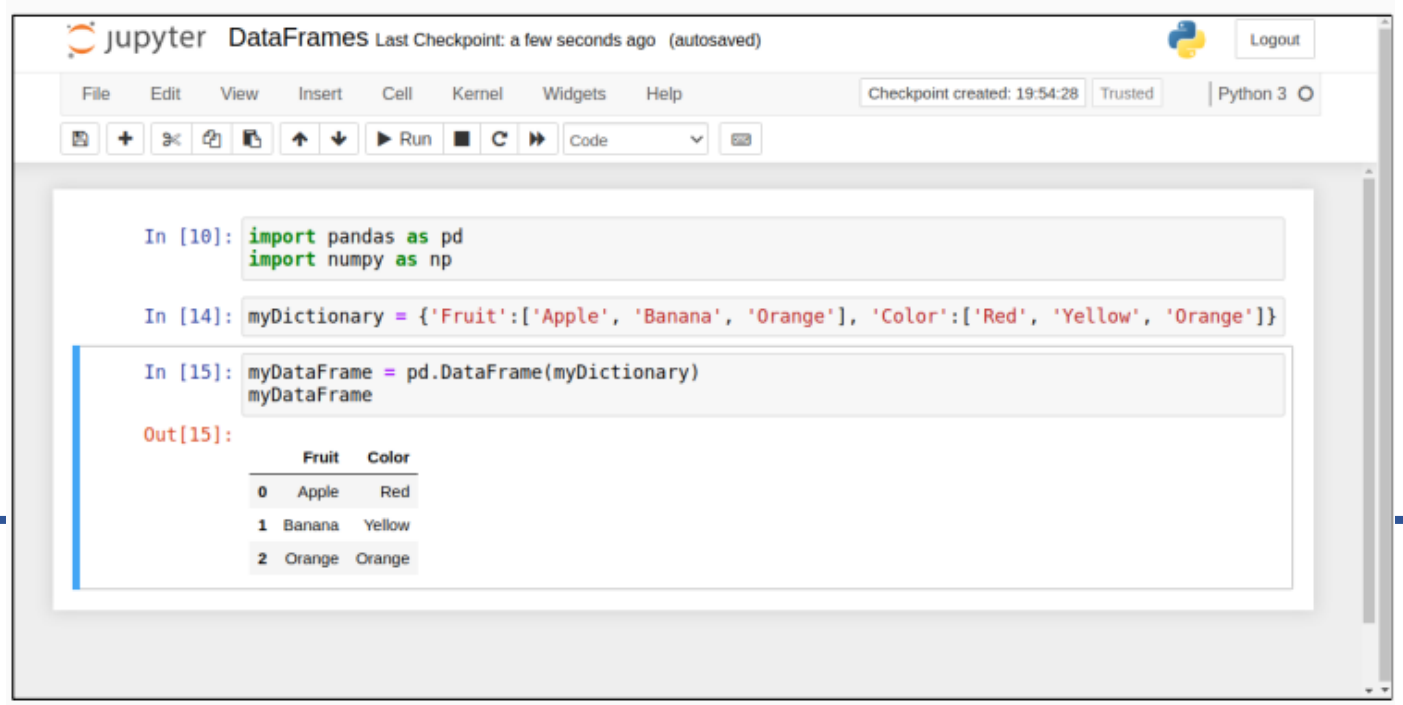
Création de DataFrame

2. Utilisation des dict :

On passe le dictionnaire en argument à la fonction pandas.DataFrame() qui nous renvoie un DataFrame de données.

Attention :

- L'attribution du paramètre « columns » n'est pas nécessaire car Pandas le récupère automatique des clés du dictionnaire



Création de DataFrame

3. Chargement de données d'un fichier CSV :

Nous pouvons également charger un fichier csv en tant que DataFrame dans Pandas en utilisant la fonction pandas.read_csv().

Attention :

- Par défaut, chaque valeur de la première ligne du fichier csv devient un libellé de colonne.

=> (Pour ne pas appliquer la première ligne en tant qu'étiquettes des colonnes, il faudra utiliser le paramètre « header = None »)

- Le séparateur par défaut pour les fichiers csv est ',. Si un autre séparateur est utiliser, il faudra ajouter le paramètre « sep = ';' » par exemple.
- Pandas attribue automatiquement des étiquettes de ligne numériques à chaque ligne du DataFrame

Création de DataFrame

Exemple :

```
In [11]: df = pd.read_csv('cereals.csv')
         df
```

Out[11]:

	name	calories	protein	vitamins	rating
0	100% Bran	70	4	25	68.402973
1	100% Natural Bran	120	3	0	33.983679
2	All-Bran	70	4	25	59.425505
3	All-Bran with Extra Fiber	50	4	25	93.704912
4	Almond Delight	110	2	25	34.384843
5	Apple Cinnamon Cheerios	110	2	25	29.509541
6	Apple Jacks	110	2	25	33.174094
7	Basic 4	130	3	25	37.038562
8	Bran Chex	90	2	25	49.120253
9	Bran Flakes	90	3	25	53.313813

```
df = pd.read_csv('cereals.csv',header=None)
df
```

	0	1	2	3	4
0	name	calories	protein	vitamins	rating
1	100% Bran	70	4	25	68.402973
2	100% Natural Bran	120	3	0	33.983678999999995
3	All-Bran	70	4	25	59.425505
4	All-Bran with Extra Fiber	50	4	25	93.704912
5	Almond Delight	110	2	25	34.384843
6	Apple Cinnamon Cheerios	110	2	25	29.509541
7	Apple Jacks	110	2	25	33.174094000000004
8	Basic 4	130	3	25	37.038562
9	Bran Chex	90	2	25	49.12025300000001
10	Bran Flakes	90	3	25	53.313813

Création de DataFrame

3. Chargement de données d'un fichier Excel :

Nous pouvons également charger un fichier excel en tant que DataFrame dans Pandas en utilisant la fonction pandas.read_excel().

Attention :

- Par défaut, chaque valeur de la première ligne du fichier excel devient un libellé de colonne.

=> (Pour ne pas appliquer la première ligne en tant qu'étiquettes des colonnes, il faudra utiliser le paramètre **« header = None »)**

- Pandas attribue automatiquement des étiquettes de ligne numériques à chaque ligne du DataFrame
- Si fichier excel avec plusieurs feuilles, il faudra préciser la feuille voulue avec le paramètre **« sheet_name = 'feuil1' »,** sinon la première feuille sera lue par défaut

Création de DataFrame

Exemple :

```
df = pd.read_excel('cereals_2.xlsx', sheet_name='TOP10')
df
```

	name	calories	protein	vitamins	rating
0	Cheerios	54	5	25.0	55.254580
1	Frosted Flakes	55	2	NaN	44.248960
2	Lucky Charms	140	8	NaN	NaN
3	Corn Flakes	85	9	25.0	NaN
4	Wheaties	56	4	22.0	88.597410
5	Froot Loops	82	6	25.0	78.148540
6	Raisin Bran	12	3	25.0	74.144265
7	Special K	24	0	NaN	NaN
8	Cocoa Puffs	82	14	3.0	NaN
9	Honey Nut Cheerios	43	8	1.5	40.847130

Création de DataFrame

Autres possibilités :

Data Description	Reader	Writer
CSV	read_csv	to_csv
JSON	read_json	to_json
HTML	read_html	to_html
Local clipboard	read_clipboard	to_clipboard
MS Excel	read_excel	to_excel
OpenDocument	read_excel	
HDF5 Format	read_hdf	to_hdf
Feather Format	read_feather	to_feather
Parquet Format	read_parquet	to_parquet
Msgpack	read_msgpack	to_msgpack
Stata	read_stata	to_stata
SAS	read_sas	
ython Pickle Format	read_pickle	to_pickle
SQL	read_sql	to_sql
Google Big Query	read_gbq	to_gbq

Modification d’index

Nous pouvons définir l'une des colonnes existantes comme la nouvelle colonne d'index du DataFrame en utilisant la fonction .set_index().

Exemple :

```
df.set_index('name')
```

name	calories	protein	vitamins	rating
Cheerios	54	5	25.0	55.254580
Frosted Flakes	55	2	NaN	44.248960
Lucky Charms	140	8	NaN	NaN
Corn Flakes	85	9	25.0	NaN
Wheaties	56	4	22.0	88.597410
Froot Loops	82	6	25.0	78.148540
Raisin Bran	12	3	25.0	74.144265
Special K	24	0	NaN	NaN
Cocoa Puffs	82	14	3.0	NaN
Honey Nut Cheerios	43	8	1.5	40.847130

Inplace

N'oubliez pas que la plupart des fonctions dans Pandas ne modifient pas le DataFrame original. Dans la section précédente, nous avons modifié la colonne d'index de notre DataFrame. Si nous imprimons à nouveau notre DataFrame, nous constatons que le DataFrame original n'a pas changé :

```
df
```

	name	calories	protein	vitamins	rating
0	Cheerios	54	5	25.0	55.254580
1	Frosted Flakes	55	2	NaN	44.248960
2	Lucky Charms	140	8	NaN	NaN
3	Corn Flakes	85	9	25.0	NaN
4	Wheaties	56	4	22.0	88.597410
5	Froot Loops	82	6	25.0	78.148540
6	Raisin Bran	12	3	25.0	74.144265
7	Special K	24	0	NaN	NaN
8	Cocoa Puffs	82	14	3.0	NaN
9	Honey Nut Cheerios	43	8	1.5	40.847130

Inplace

Deux méthodes possibles :

```
df.set_index('name', inplace=True)
```

name	calories	protein	vitamins	rating
Cheerios	54	5	25.0	55.254580
Frosted Flakes	55	2	NaN	44.248960
Lucky Charms	140	8	NaN	NaN
Corn Flakes	85	9	25.0	NaN
Wheaties	56	4	22.0	88.597410
Froot Loops	82	6	25.0	78.148540
Raisin Bran	12	3	25.0	74.144265
Special K	24	0	NaN	NaN
Cocoa Puffs	82	14	3.0	NaN
Honey Nut Cheerios	43	8	1.5	40.847130

Utilisation de Inplace=True

```
df = df.set_index('name')
```

	name	calories	protein	vitamins	rating
0	Cheerios	54	5	25.0	55.254580
1	Frosted Flakes	55	2	NaN	44.248960
2	Lucky Charms	140	8	NaN	NaN
3	Corn Flakes	85	9	25.0	NaN
4	Wheaties	56	4	22.0	88.597410
5	Froot Loops	82	6	25.0	78.148540
6	Raisin Bran	12	3	25.0	74.144265
7	Special K	24	0	NaN	NaN
8	Cocoa Puffs	82	14	3.0	NaN
9	Honey Nut Cheerios	43	8	1.5	40.847130

Ecrasement de valeur

Examination des données

Fonctions utiles :

- df.head(n) => retourne les n premières lignes du DataFrame (par défaut n = 5)
- df.tail(n) => retourne les n dernières lignes du DataFrame (par défaut n = 5)
- df.shape => retourne les dimensions du DataFrame
- df.info() => retourne les types de données de chaque colonne, le nombre d'éléments non nuls et l'utilisation de la mémoire
- df.columns => retourne les libellés des colonnes du DataFrame.
- df.dtypes => retourne le type de données de chaque colonne dans le DataFrame
- df.describe() => retourne des statistiques descriptives des colonnes numériques dans le DataFrame, telles que le nombre, la moyenne, l'écart-type, le minimum, le maximum et les percentiles.

Selection des données

Utilisation des crochets [] pour sélectionner les lignes :

- Nous pouvons utiliser l'opérateur crochet ([ ]) pour découper les lignes du DataFrame.
- Nous passons un indice de début (inclusif) et un indice de fin (exclusif) à l'opérateur crochet ([ ]) pour découper les lignes du DataFrame.

```
df[1:4]
```

	name	calories	protein	vitamins	rating
1	Frosted Flakes	55	2	NaN	44.24896
2	Lucky Charms	140	8	NaN	NaN
3	Corn Flakes	85	9	25.0	NaN

Selection des données

Utilisation des crochets [] pour sélectionner les lignes:

N'oubliez pas que l'opérateur [ ] fonctionne sur la position des lignes et non sur les libellés de lignes.

Par exemple, dans le cas suivant, les libellés de lignes sont des chaînes de caractères. Mais nous passons les positions des lignes que nous voulons découper.

```
df
```

name	calories	protein	vitamins	rating
Cheerios	54	5	25.0	55.254580
Frosted Flakes	55	2	NaN	44.248960
Lucky Charms	140	8	NaN	NaN
Corn Flakes	85	9	25.0	NaN
Wheaties	56	4	22.0	88.597410
Froot Loops	82	6	25.0	78.148540
Raisin Bran	12	3	25.0	74.144265
Special K	24	0	NaN	NaN
Cocoa Puffs	82	14	3.0	NaN
Honey Nut Cheerios	43	8	1.5	40.847130

```
df[1:4]
```

	name	calories	protein	vitamins	rating
1	Frosted Flakes	55	2	NaN	44.24896
2	Lucky Charms	140	8	NaN	NaN
3	Corn Flakes	85	9	25.0	NaN

Selection des données

Utilisation des crochets [] pour sélectionner les lignes:

Nous pouvons également passer une liste de booléens à l'opérateur [ ].
Nous obtenons toutes les lignes du DataFrame pour lesquelles l'élément correspondant dans la liste est **True**.
Les lignes du DataFrame pour lesquelles l'élément correspondant dans la liste est **False** sont ignorées.

Remarque : Le DataFrame d'origine reste inchangé.

```
In [71]: df
```

Out[71]:

	name	calories	protein	vitamins	rating
0	100% Bran	70	4	25	68.402973
1	100% Natural Bran	120	3	0	33.983679
2	All-Bran	70	4	25	59.425505
3	All-Bran with Extra Fiber	50	4	25	93.704912
4	Almond Delight	110	2	25	34.384843

```
In [75]: thirdRow = [False, False, True, False, False]
         df[thirdRow]
```

Out[75]:

	name	calories	protein	vitamins	rating
2	All-Bran	70	4	25	59.425505

Question 1

Considérons le DataFrame donné appelé df. Lequel des éléments suivants nous donnera les lignes de la 5ème ligne à la 10ème du DataFrame ?

a) df[5:10]
b) df[5:11]
c) df[4:10]
d) df[4:11]

Question 1

Considérons le DataFrame donné appelé df. Lequel des éléments suivants nous donnera les lignes de la 5ème ligne à la 10ème du DataFrame ?

a) df[5:10]
b) df[5:11]
c) df[4:10]
d) df[4:11]

Réponse :

La 5ème ligne est la ligne dont l'index est 4 en terme de position.
La 10ème ligne est la ligne dont l'index est 9 en terme de position.

a) df[5:10]
b) df[5:11]
c) df[4:10]
d) df[4:11]

Selection des données

Utilisation des crochets [] pour sélectionner les colonnes:

Nous pouvons également utiliser l'opérateur crochets ([ ]) pour indexer les colonnes du DataFrame.

- L'indexation d'une seule colonne renvoie une Series.
- L'indexation d'une liste de colonnes renvoie un DataFrame.
- N'oubliez pas que pour indexer les colonnes, nous passons leurs étiquettes à l'opérateur [ ] et non leurs positions.

```
df[['name', 'rating']]
```

	name	rating
0	100% Bran	68.402973
1	100% Natural Bran	33.983679
2	All-Bran	59.425505
3	All-Bran with Extra Fiber	93.704912
4	Almond Delight	34.384843

Notebook 2 : Découvrir les DataFrames